

Study Case 2 — Hazır Modellerle Video Analizi: Takip (Tracking) + Olay Mantığı (Counting)

Bu çalışmada, açık bir video veri seti üzerinde hazır (pretrained) bir algılama + takip (tracking) yaklaşımı kullanarak, üstüne gerçek hayatta sık ihtiyaç duyulan bir “olay mantığı” kurmanı bekliyoruz: videoda belirli bir yerden geçen kişi sayısını (giriş/çıkış gibi) güvenilir şekilde çıkarmak.

Ağır model eğitimi beklemiyoruz. Hazır modelleri kullanabilirsin. Çözümün, makul sürede CPU’da çalışabilir olması tercih edilir (Colab serbest).

Bu case yalnızca MOT17 açık veri seti ile yapılacaktır. Alt küme seçimini (hangi sequence’ler üzerinde çalışacağın) sen yapabilirsin. Ancak seçtiğin sequence’leri repo içinde açıkça belirtmelisin.

Bu çalışma sonunda, bir video üzerinde şuna benzer çıktılar üretmeni bekliyoruz (yönlendirme amaçlıdır);

- Videoda görülen kişilere ait track çıktısı (ID bazlı takip bilgisi)
- Bir “sanal çizgi/alan” tanımına göre giriş/çıkış sayımı gibi olay çıktısı
- Sonuçları hızlı anlamamızı sağlayacak overlay demo video/GIF
- Sonuçları sayısal olarak özetleyen metrik/tablolara ve kısa yorum

Burada görev tarif etmiyoruz. Problemden doğru çıkarımı yapman ve çözümü kendin kurgulamalısın.

İpuçları

Veri setlerinde ve gerçek hayatta karşılaşılan bazı zorluklar:

- Occlusion (örtüşme): kişi kaybolur/geri gelir → ID karışabilir
- Kalabalık sahneler: yanlış eşleştirme ve ID switch artar
- Kısa süreli miss: algılama kaçırınca takip kopabilir
- Giriş/çıkış sayımı: aynı kişi birden fazla kez sayılabilir veya hiç sayılmayabilir

Bu nedenle güçlü bir çözümde genellikle şunlar öne çıkar:

- Olay mantığını (counting) track ID davranışına dayanarak sağlamlaştırmak
- Hataları örnekleyip (görsel olarak) neden oluştuğunu açıklamak
- Parametre/konfigürasyon seçimlerini gerekçelendirmek (eşikler, buffer mantığı vb.)
- Farklı sequence türlerinde davranış karşılaştırmak

Bizim için kritik olan sonuçların doğruluğu kadar, çözümün neden iyi/kötü çalıştığını anlatabilmen.

Analiz Beklentisi (Biz neye bakacağız?)

- Hangi sahnelerde sayım doğru, hangilerinde bozuluyor?
- Bozulmanın kaynağı detection mı, tracking mi, yoksa olay mantığı mı?
- En az birkaç somut örnekle (overlay görüntülerle) hata tiplerini gösterebiliyor musun?
- Çözümün tekrarlanabilir mi ve çıktıları anlaşılır mı?

Not: Tracking metrikleri ve counting metrikleri için “tek bir doğru set” dayatmıyoruz. Önemli olan metriklerin seçimini mantıklı bir çerçeve ile sunman ve çıktıları okunur hale getirmen.

Teslim Formatı

1) Git Repository

Çalışmanı bir Git (Github/Gitlab vs.) reposu olarak paylaşmalısın. Repo temiz, çalıştırılabilir ve tekrar üretilebilir olmalı.

2) Repo içinde zorunlu dosyalar

- **README.md**
 - Projenin kısa özeti
 - Kurulum ve çalıştırma talimatı
 - Kullandığın dataset sequence’leri
 - Çizgi/alan tanımını nasıl yaptığını kısaca belirt (konum, mantık)
 - Üretilen çıktılar nerede oluşuyor? (klasör yapısı)
- **requirements.txt** veya **environment.yml**
- Kod yapısı (tercihen **src/** altında)
 - Notebook kullanabilirsin
- **report.pdf** (öneri: 2–4 sayfa)
 - Yaklaşımın kısa özeti
 - Örnek overlay çıktılar (ID + çizgi + sayım olayı görünür)
 - Seçtiğin metrikler/tablolara ve yorum
 - Zorlandığın örnekler + kısa hata analizi
 - “İyileştirme yapsaydım ne denerdim?” gibi kısa bir bölüm

3) Çıktı (repo içinde)

Repo içinde (veya Git release / link ile) şu tip çıktıları bekliyoruz:

- En az 1–2 sequence için kısa **demo video/GIF** (track ID + çizgi + olay işaretleri)
- Sonuç özetini gösteren **grafik/tablolara** (PNG veya PDF)
- Olay çıktısı (örn. **counts.json** / **events.csv** gibi)
- (Varsa) takip çıktısı dosyası (örn. MOT formatına benzer bir **tracks.txt**)

4) Çalıştırılabilirlik standardı

Tercihimiz:

- Tek bir giriş noktası (örn. `python run.py ...`)
- Çıktıları belirli bir klasöre yazma (`outputs/` gibi)